

Contagem paralela e verificável de números primos

Medição do ganho de processos sobre threads em trabalho limitado por processador

Equipe

Cauê Barroso · 24070033
César Ribeiro · 24070011
Augusto Pereira · 24070005

Turma

CC6MA
Data
22/09

Professor

Fábio Rocha de Araújo

Repositório

(link no ambiente virtual)

1 O problema

O programa conta os primos do intervalo `[2, N]` e devolve três valores — **contagem**, **maior primo** e **soma dos primos** — com $N = 20.000.000$.

Por que se divide. Decidir se um número é primo não usa o resultado de nenhum outro: não há dependência de dados nem de ordem. A unidade de trabalho é um **bloco de 10.000 inteiros consecutivos**; para $N = 20$ milhões são **2.000 blocos independentes**.

Volume. 20 milhões de candidatos, dos quais 1.270.607 são primos. O teste é **divisão sucessiva em Python puro** — escolha deliberada: queríamos trabalho *de processador*, e não uma chamada a biblioteca em C que liberasse a GIL e embaralhasse a comparação entre threads e processos. A versão sequencial leva **128,62 s** — minutos, não segundos.

Verificação, em três camadas. (1) **Assinatura de três campos:** acertar a contagem por acaso é plausível; acertar os três trocando um primo por outro, não. (2) **Algoritmo independente:** um Crivo de Eratóstenes calcula a mesma resposta por outro caminho em 0,65 s — conferir apenas sequencial contra paralelo não pegaria um erro no teste de primalidade, que é comum aos dois. (3) **Valor externo:** $\pi(20.000.000) = 1.270.607$ confere com a literatura. Além disso, a versão paralela é repetida cinco vezes com a mesma entrada e precisa devolver o mesmo trio (§3.4).

2 Estratégia de paralelização

Paralelismo de dados: todos os trabalhadores executam a mesma operação sobre fatias diferentes do mesmo domínio. Não há etapas distintas que virassem tarefas.

Processos, não threads. O laço quente é divisão inteira em Python puro: **não há espera** — nem disco, nem rede, nem banco. O trabalho é **limitado por processador**. Em CPython a GIL permite que só uma thread execute bytecode por vez, então threads não somam tempo de CPU. Não tomamos isso como artigo de fé: medimos as duas versões, com o mesmo algoritmo e a mesma entrada (§6.4). Processos resolvem porque cada um tem seu interpretador e sua GIL, e é escalonado num núcleo de verdade.

Divisão do trabalho: fila com balanceamento dinâmico. Os 2.000 blocos vão para uma `multiprocessing.Queue` e cada trabalhador puxa o próximo ao terminar o anterior. Isso importa porque **blocos de tamanho igual não têm custo igual**: testar um número perto de 20 milhões percorre divisores até ~4.472, contra poucas dezenas perto de zero — o fim do intervalo é muito mais caro que o começo. Com quatro fatias estáticas para quatro processos, quem recebesse a última seguraria os outros três ociosos no fim. Com $T \gg W$ (2.000 blocos para até 8 trabalhadores), a fila reequilibra sozinha, sem que o programa precise estimar o custo de cada bloco.

3 A seção crítica e a primitiva que a protege

Estado compartilhado: três inteiros de 64 bits em memória compartilhada entre processos (`multiprocessing.RawValue('q')`) — `contagem`, `maior` e `soma`. Os W trabalhadores escrevem nos três.

A **seção crítica** é a fusão do parcial local no global, e nada além disso (`src/paralelo_processos.py`):

```
def _dobrar_no_estado(contagem, maior, soma, local):
    contagem.value += local.contagem          # leitura-modificação-escrita
    if local.maior_primo > maior.value:        # leitura-modificação-escrita
        maior.value = local.maior_primo
    soma.value += local.soma                  # leitura-modificação-escrita

def trabalhador(fila, contagem, maior, soma, trava, usar_trava):
    while True:
        tarefa = fila.get()
        if tarefa is None:
            return
        inicio, fim = tarefa
        local = resumo_intervalo(inicio, fim)  # PESADO – fora da trava
        with trava:                             # seção crítica
            _dobrar_no_estado(contagem, maior, soma, local)
```

Nenhuma das três linhas é atômica: cada uma lê, calcula e escreve de volta. Dois processos executando isso ao mesmo tempo podem ler o mesmo valor antigo, e a escrita de um apaga a do outro — a soma de um bloco inteiro se perde.

O que está fora da trava importa tanto quanto o que está dentro. A chamada `resumo_intervalo`, que consome mais de 99 % do tempo, fica fora. Uma versão com a trava em volta do laço seria igualmente correta, mas serializaria o programa — um processo por vez, $\text{speedup} \approx 1$. Aqui a trava protege três somas de microssegundos, adquiridas 2.000 vezes na execução inteira.

Por que Lock, e não semáforo ou monitor. O que o problema pede é **exclusão mútua pura**. Semáforo serve para limitar um recurso **contável** — não temos contagem a limitar. Monitor serve quando há **condição de espera** a sinalizar — nenhum trabalhador espera por estado de outro. O mutex é a primitiva mínima que resolve, e a mínima é a certa.

3.4 Prova de que não há condição de corrida

Teste 1 — estabilidade do resultado real (`verificar.py`): a versão paralela roda cinco vezes com a mesma entrada; os três campos vêm idênticos nas cinco, e idênticos ao sequencial e ao crivo. Perder atualizações é erro **não determinístico** — se houvesse corrida, a contagem variaria entre execuções.

Teste 2 — o contraste (`demo_corrida.py`): o teste 1 mostra que está certo, mas não que é a trava que faz estar certo. Seis processos incrementam 100.000 vezes cada um o mesmo contador; o correto é 600.000.

EXECUÇÃO	SEM TRAVA	PERDIDOS	COM TRAVA
1	373.508	226.492	600.000
2	275.580	324.420	600.000
3	272.174	327.826	600.000

Sem a trava perde-se ~45 % dos incrementos, e **o número muda a cada execução** — numa segunda bateria deu 369.254 / 352.851 / 337.311, seis valores errados e todos diferentes, enquanto a coluna com trava deu 600.000 nas seis. O resultado estável vem da primitiva, não de sorte de escalonamento.

3.5 Troca de mensagem e carimbo lógico

A aplicação **tem** troca de mensagem: o coordenador envia blocos aos trabalhadores por uma fila e recebe registros de evento por outra. Como processos separados não compartilham relógio confiável, a ordem vem do **carimbo lógico de Lamport** (`src/lamport.py`). Regras: evento local $L+1$; envio $L+1$, e a mensagem viaja com L ; recebimento $\max(L, L_{\text{msg}})+1$. Cada registro contém **quem** ($C_0, T_1 \dots T_n$), **o quê** (`ENVIA_BLOCO`, `RECEBE_BLOCO`, `CALCULA`, `ENTRA_SC`, `SAI_SC`), **o carimbo** e **sobre o quê** (o bloco `[inicio, fim)` ou `estado_global`). O log mostra a causalidade preservada — o coordenador envia com carimbo 1 e quem recebe registra 2, nunca o contrário.

Limite reconhecido: carimbos de Lamport dão ordem **parcial**. Eventos concorrentes podem receber o mesmo carimbo, e carimbo maior não implica dependência causal; detectar concorrência exigiria relógios vetoriais.

4 Arquitetura e provisionamento na nuvem

Esta seção registra as decisões de provisionamento da equipe e a razão de cada uma. O ambiente usado é o **AWS Academy (Learner Lab)**, que restringe a região e os portes de instância disponíveis; onde isso afeta uma decisão, está anotado. A instância é criada por execução (última linha da tabela) e demonstrada ao vivo no console. **A medição relatada na seção 5 foi feita na máquina descrita ali**, e não na instância.

Zonas de disponibilidade	Uma única zona — SLA de 99,5 %
Região	A disponibilizada pelo ambiente acadêmico
Instância	1, família compute-optimized, o maior porte disponível — alvo: 8 vCPU / 4 núcleos físicos
Armazenamento	EBS gp3, 20 GB (disco raiz)
Portas de entrada	22/tcp ← <IP da equipe>/32 · 8000/tcp ← 0.0.0.0/0
Ciclo de vida	Criada por execução

Zona única e SLA. A decisão é operar com **instância isolada em uma única zona**, e com isso o compromisso que passa a valer é o de **99,5 % — 216 minutos** de indisponibilidade admitida em 30 dias. Os **99,99 %** (4,3 minutos) exigiriam instâncias em **duas ou mais zonas**. Aceitamos 99,5 % conscientemente: a carga é um **lote** de poucos minutos, não um serviço contínuo; se a instância cair, repete-se a execução. E há razão de método — a medição exige sequencial e paralelo **na mesma máquina**, então duas zonas significariam duas instâncias, o dobro do custo e nenhum ganho de medição. A região é a disponibilizada pelo ambiente acadêmico, que não permite escolhê-la; o compromisso de SLA independe dela.

Família e porte. A família `c` (compute-optimized) é a adequada, porque o gargalo é CPU. Descartamos a família `t` (burstable) por motivo concreto: ela entrega créditos de CPU que se esgotam, e uma medição de minutos terminaria com o desempenho caindo no meio — o tempo medido não seria o do algoritmo. Memória não é gargalo: o programa não guarda a lista de primos, só acumula três inteiros, e usa poucos MB. O porte-alvo é de 8 vCPU (4 núcleos físicos); como o Learner Lab limita os portes, usamos o maior disponível, e é o número de núcleos **físicos** efetivo que determina até qual `W` vale medir — abaixo de dois núcleos físicos não há ganho de paralelismo a observar (§6.3c).

Entrada e saída. Não há arquivo de entrada — a entrada é o parâmetro `N`, e os blocos são gerados em memória. A saída são `resultados.json` (~4 KB), `medicao_console.txt` (~2 KB) e `eventos.log` (poucos KB), no volume EBS. **Não usamos S3:** com dados dessa ordem ele só acrescentaria latência de rede dentro do laço de medição, permissões de IAM e mais um ponto de falha.

Portas. Por que a porta administrativa não está aberta para 0.0.0.0/0 : SSH exposto à internet recebe varredura automatizada e força bruta continuamente; com a porta administrativa pública, qualquer credencial fraca ou vazada deixa de ser incidente contido e passa a ser **controle total da instância**. A superfície de administração só precisa ser alcançável de onde a equipe administra — daí o `/32`. Isso não é só intenção documentada: `provisionar_aws.sh` **aborta** se não descobrir o IP da equipe, em vez de cair para `0.0.0.0/0`, e ao final **confere as regras e falha** se achar `0.0.0.0/0` na porta 22. A 8000 é aberta por ser a **porta do serviço** — uma página somente leitura, sem autenticação a proteger e sem rota que altere estado (por isso também HTTP simples: não há segredo em trânsito).

Ciclo de vida. Criada por execução (`provisionar` → `executar` → `destruir`). A carga dura minutos; manter a máquina ligada fora dessa janela é custo sem uso. Como não há estado a preservar entre execuções, recriar não custa nada além da inicialização. No Learner Lab o ambiente reforça essa decisão: a sessão do laboratório encerra e para as instâncias automaticamente, de modo que o ciclo por execução é o único regime possível.

5 Medição

Protocolo. As duas versões rodam **na mesma máquina**, com **a mesma entrada**, na **mesma invocação** de `benchmark.py`. Cada configuração é executada **três vezes** e reportada pela **mediana**, cronometrada com `time.perf_counter()`. O resultado de toda execução é conferido contra o crivo **antes** de o tempo ser aceito: tempo bom com resultado errado não conta.

Onde foi medido. Os tempos abaixo vêm da **máquina de desenvolvimento da equipe**, identificada a seguir — não da instância na nuvem. O porte de instância disponível no ambiente acadêmico não oferece os núcleos físicos necessários para medir ganho de paralelismo com significado, e medir ali produziria um número que reflete o limite do ambiente, não o do programa. A exigência que de fato importa para o speedup está atendida: sequencial e paralelo foram medidos **na mesma máquina, com a mesma entrada, na mesma invocação**. Todas as execuções foram feitas com a máquina ociosa, sem outra carga concorrente.

Ambiente: Windows 11, AMD Ryzen, **6 núcleos físicos** / 12 lógicos, CPython 3.14.4, N = 20.000.000, blocos de 10.000. Verificação: 1.270.607 primos, maior 19.999.999, soma 12.272.577.818.052 — **as 15 execuções produziram exatamente esse trio**.

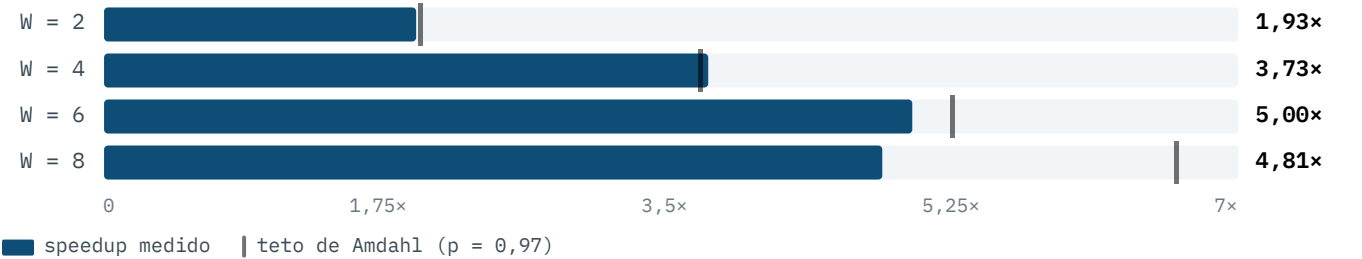
VERSÃO	W	EXECUÇÕES (S)	MEDIANA	SPEEDUP	EFIC.	TETO AMDAHL	P MEDIDO
sequencial	1	128,62 · 128,14 · 131,81	128,62 s	1,00×	1,00	—	—
processos	2	66,77 · 67,75 · 66,55	66,77 s	1,93×	0,96	1,94×	0,962
processos	4	34,34 · 34,47 · 34,56	34,47 s	3,73×	0,93	3,67×	0,976
processos	6	25,40 · 25,87 · 25,74	25,74 s	5,00×	0,83	5,22×	0,960
processos	8	25,29 · 26,92 · 26,73	26,73 s	4,81×	0,60	6,61×	0,905
threads	4	140,53 · 139,13 · 146,85	140,53 s	0,92×	0,23	—	—

Melhor resultado: 128,6 s → 25,7 s, speedup de 5,00× com W = 6. E repare no que acontece depois: **W = 8 é mais lento que W = 6** (4,81× contra 5,00×). A máquina tem seis núcleos físicos, e o sétimo e o oitavo trabalhadores não têm onde rodar.

O protocolo é reproduzível: repetida a medição em outra máquina (a instância EC2, via `scripts/executar_na_instancia.sh`), as tabelas são regeradas por `python src/tabelas.py` a partir do JSON que o próprio `benchmark.py` grava.

6 Análise: o medido contra o previsto

6.1 O medido acompanha o teto — até acabarem os núcleos físicos



W	PREVISTO (P = 0,97)	MEDIDO	FRAÇÃO DO TETO	EFICIÊNCIA
2	1,94×	1,93×	99 %	0,96
4	3,67×	3,73×	102 %	0,93
6	5,22×	5,00×	96 %	0,83
8	6,61×	4,81×	73 %	0,60

A tabela se parte em duas metades. **Até W = 6 — que é exatamente o número de núcleos físicos — o medido acompanha o previsto de perto**, entre 96 % e 102 %. De W = 6 para W = 8 a coisa inverte: a fração do teto cai para 73 % e, mais do que isso, **o speedup diminui em valor absoluto** (5,00× → 4,81×). Acrescentar dois trabalhadores deixou o programa mais lento.

Esse é o achado central da medição, e ele não precisa de interpretação: o ganho é limitado pelo número de **núcleos físicos**, e pedir mais paralelismo do que a máquina tem não é neutro — custa.

6.2 A estimativa de p estava certa

A métrica de **Karp-Flatt** inverte a pergunta: parte do speedup *medido* e calcula qual fração paralelizável o explicaria.

W	SPEEDUP MEDIDO	P MEDIDO
2	1,93×	0,962
4	3,73×	0,976
6	5,00×	0,960
8	4,81×	0,905

Até $W = 6$, o p medido fica entre **0,96 e 0,98** — em torno dos 0,97 que estimamos. A estimativa da equipe estava correta e, em $W = 4$, até ligeiramente conservadora: é por isso que ali o medido (3,73×) **passa** o teto calculado com 0,97 (3,67×). Não é violação da lei de Amdahl; é sinal de que o p verdadeiro naquele ponto é 0,976, e não 0,97.

A queda para 0,905 em $W = 8$ também não indica que o algoritmo mudou. O que mudou foi o denominador do modelo: Amdahl supõe n unidades de execução **independentes**, e a oitava não existe nesta máquina. O modelo, forçado a explicar um speedup pior com um n maior, devolve a única resposta que pode — uma fração serial maior.

6.3 O que limitou o ganho

(a) Espera na seção crítica — descartada. A trava é adquirida 2.000 vezes na execução inteira, e cada seção crítica são três somas de inteiros: num total de 34 s, menos de 0,1 %. A seção crítica **não** é o gargalo — e é justamente esse o efeito de tê-la mantido pequena. A eficiência de **0,93 em $W = 4$** confirma: quase nada se perde em coordenação.

(b) Custo fixo de criar processos e mover blocos — pequeno aqui, decisivo em volumes menores. O `spawn` reinicia o interpretador em cada processo, e os blocos são serializados pela fila. Esse custo é ~constante em segundos, então o que muda é o tamanho do trabalho que o dilui. Com o mesmo código e o mesmo $W = 4$:

N	TEMPO SEQUENCIAL	SPEEDUP COM $W = 4$
1.000.000	1,85 s	2,44×
20.000.000	128,62 s	3,73×

É por isso que a exigência de que o sequencial leve *minutos* não é burocracia: em segundos, a medição mediria sobretudo o custo de partida, e concluiria que o programa escala mal quando o que escala mal é o experimento.

(c) Núcleos físicos — o limite dominante, e desta vez medido diretamente. A eficiência cai de 0,96 ($W = 2$) para 0,93 ($W = 4$), 0,83 ($W = 6$) e 0,60 ($W = 8$). Até seis trabalhadores a queda é suave — é o custo de coordenação crescendo devagar. De seis para oito ela desaba, porque não há mais núcleo para ocupar: os dois trabalhadores extras passam a dividir núcleos com os outros seis, e o SMT não compensa o que a troca de contexto custa. Daí **o $W = 8$ ser mais lento que $W = 6$** .

Isso também mostra que **comparar $W = 8$ com o teto de 6,61× é injusto com o modelo**: esse teto pressupõe oito unidades independentes, e há seis. O teto honesto para esta máquina é Amdahl com $n = 6$ — **5,22×** —, e os 5,00× medidos em $W = 6$ são **96 %** dele.

(d) Frequência do processador — fator provável, não isolado. Com um núcleo ativo o processador opera em turbo mais alto do que com todos carregados, o que penaliza o speedup porque o denominador da conta (o tempo sequencial) é obtido na condição mais favorável. **Não isolamos esse efeito** — exigiria fixar a frequência no firmware —, então fica registrado como causa provável, e não como valor medido.

6.4 Threads: a evidência que justifica a escolha

Com quatro threads o programa levou **140,53 s** contra **128,62 s** do sequencial: ficou **12 segundos mais lento que sem paralelismo nenhum** (0,92×). É o previsto — a GIL permite só uma thread executando bytecode por vez, de modo que as quatro se revezam em vez de somar tempo de processador, e ainda pagam troca de contexto e disputa pela trava. O mesmo algoritmo, com processos, rende **5,00×**. É a diferença entre afirmar e demonstrar: a escolha por processos não é preferência, é o que os números exigem.

7 Limites reconhecidos

- **O ganho é limitado por núcleos físicos, não por vCPU** — cada vCPU x86 é uma *thread* de um núcleo, e duas threads do mesmo núcleo disputam as mesmas unidades de execução.
- **A medição vale para este volume** — com N muito menor, o custo de partida domina ($2,44\times$ em $N = 1$ milhão contra $3,73\times$ em 20 milhões, mesmo W).
- **Carimbos de Lamport dão ordem parcial** — preservam causalidade, não detectam concorrência; isso exigiria relógios vetoriais.
- **O checksum usa 64 bits** — a soma até 20 milhões ($1,2 \times 10^{13}$) cabe folgada; acima de $\sim 10^9$ seria preciso trocar o tipo do acumulador compartilhado.

Uso de inteligência artificial. Este trabalho foi desenvolvido com auxílio do Claude (Anthropic) na escrita do código e na redação deste relatório. As decisões de projeto foram revisadas e validadas pela equipe, e todos os tempos apresentados são medições reais, registradas em `resultados/resultados.json` e conferidas contra um algoritmo independente.